

Sprint 6 — Plan Detallado con Issues Especificadas

Proyecto: Gestor-Horarios
Fecha: 2026-07-27
Estado: Plan completo — listo para implementación

Resumen del Sprint

#	Issue	Asignado	Tipo	Prioridad	Depende de
#39	Implementacion restricciones blandas CP-SAT	Luis	Backend	High	Ninguna
#37	Tuning del modelo Solver CP-SAT	Nicole	Backend	High	#39
#41	Optimizacion y performance Solver CP-SAT	Nicole	Backend	High	#39
#36	Frontend admin module (gestión usuarios)	Dani	Frontend	High	ServicioProfesor*
#38	Pantalla de configuración del sistema (shell estético)	Paola	Frontend	Medium	Ninguna
#40	Pruebas integración front → middle → back	Manuel	QA	High	ServicioProfesor* + #36

* ServicioProfesor + ServicioDisponibilidad es tarea de implementación de Luis, no tiene issue separada.

Dependencias

```
ServicioProfesor/Disponibilidad (Luis, impl) —→ #36 (Frontend Admin) → #40 (Integration Tests)

#39 (Soft Constraints) —→ #37 (Tuning)
|                          —→ #41 (Performance)

#38 (Config Shell) —→ independiente (puede empezar inmediatamente)
```

Propuesta

Intención

Sprint 6 cierra la brecha entre persistencia de datos y el solver. El backend tiene servicios CRUD funcionales (Aula, Materias) pero falta el servicio de profesores y disponibilidad. El middleware devuelve stubs. El

frontend muestra datos estáticos. El solver tiene 8 restricciones duras pero PenalizacionConfig es código muerto. Este sprint completa la capa de datos para que el solver consuma disponibilidad real de profesores, agrega restricciones blandas, ajusta el rendimiento del solver y proporciona pantallas de administración funcionales al frontend.

Alcance

Incluido:

- ServicioProfesor + ServicioDisponibilidad (tarea de implementación, no issue separada)
- Wiring del middleware (reemplazar stubs con llamadas reales)
- Restricciones blandas del solver (usar PenalizacionConfig)
- Tuning del solver (parámetros CP-SAT)
- Optimización de performance del solver
- Módulo admin del frontend (TeacherListWidget conectado)
- Shell de configuración del sistema (puramente estético/general)
- Pruebas de integración end-to-end

Excluido:

- Autenticación de usuarios
- Control de acceso basado en roles
- Generación de reportes (exportación PDF es separada)
- Soporte multi-institución
- Configuración del solver en la UI (solo archivos de configuración backend)

Diseño Técnico

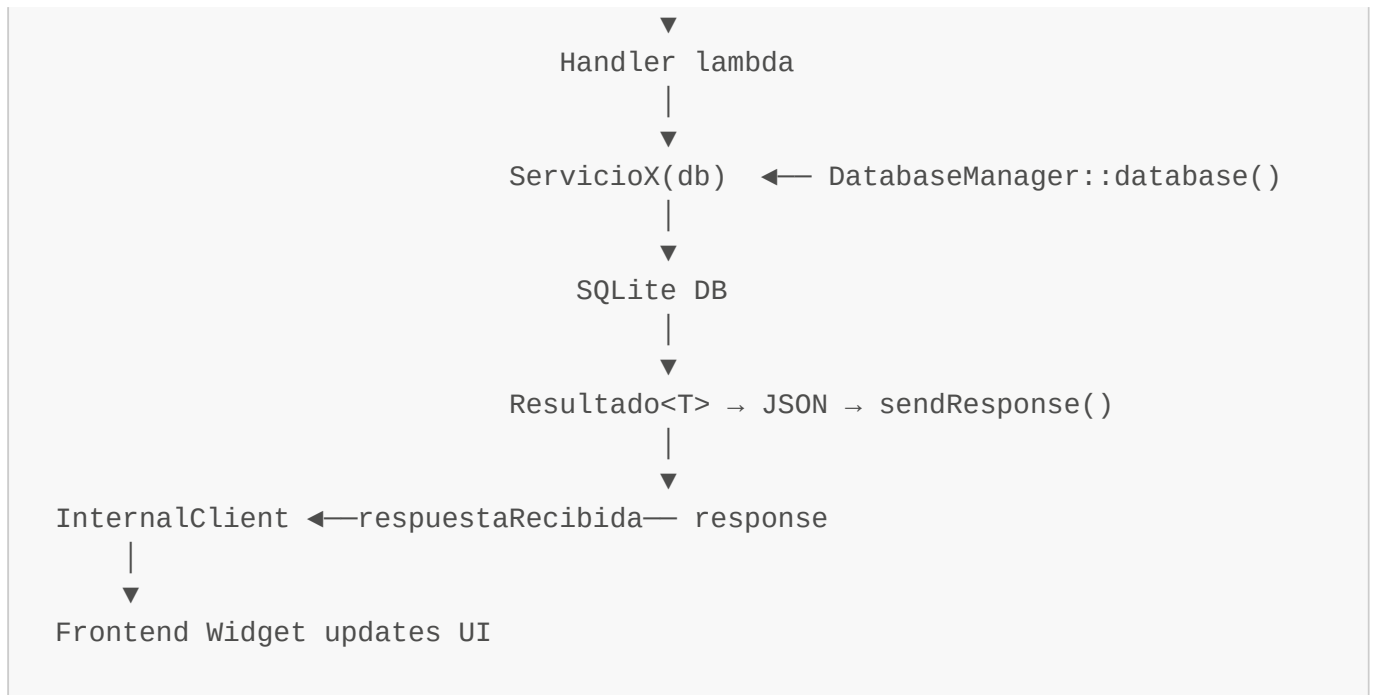
Decisiones de Arquitectura

1. **Middleware → Backend Directo:** InternalServer instancia servicios directamente (mismo proceso, SQLite compartido). Sin nueva capa IPC.
2. **PenalizacionConfig como Pesos de Restricciones Blandas:** Expandir con 3 nuevos pesos, crear BoolVars de penalización, `model.Maximize()`.
3. **Mapas Pre-calculados:** Construir QMap indexado durante creación de variables para reducir $O(n^2)$ a $O(1)$ por lookup.
4. **Config Shell Puramente Estético:** Sin configuración del solver. Solo un shell navegable con paneles placeholder para sprints futuros.

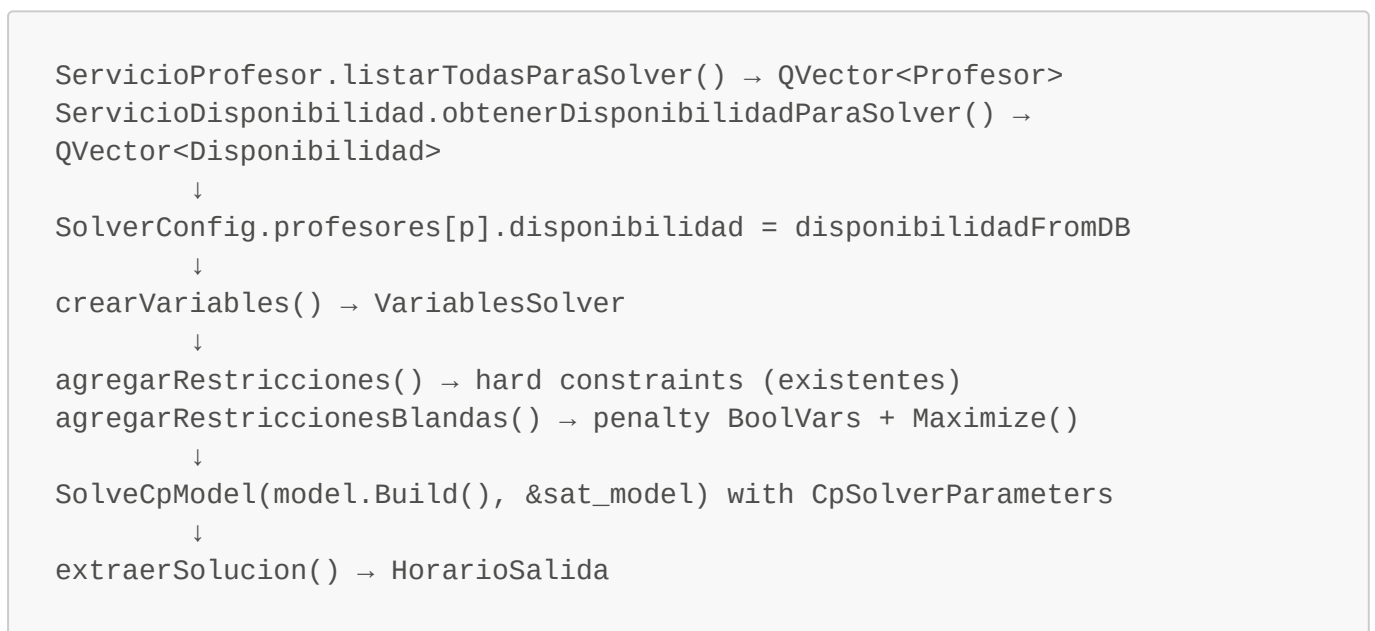
Flujo de Datos

```

Frontend Widget
  | enviarSolicitud(op, payload)
  ▼
InternalClient —QLocalSocket—► InternalServer
                               | m_rutas[op]
  
```



Flujo del Solver



Issues Detalladas

ServicioProfesor + ServicioDisponibilidad (Tarea de Implementación — Luis)

Objetivo: Implementar los servicios backend faltantes para la gestión de disponibilidad de profesores y el CRUD completo de profesores.

Contexto Técnico:

- La tabla Disponibilidad_Profesor existe en la BD pero NO tiene servicio
- NO existe ServicioProfesor (solo stubs en middleware)
- Los servicios existentes (ServicioAula, ServicioMaterias) siguen el patrón de CRUD con SQLite

- Patrón a seguir: ServicioAula.hpp/cpp (DTO + validación + Resultado + SQL)

Requisitos:

1. ServicioProfesor — CRUD Completo

- Implementar crear, obtener, listar, actualizar, eliminar en tabla Profesores
- PK es TEXT (cédula), no autoincrement
- Cascade delete: al eliminar profesor, eliminar Profesor_Materia y Disponibilidad_Profesor
- Validación: nombre no vacío, email formato válido, id no vacío
- Método `obtenerTodosParaSolver()` que retorne QVector con disponibilidad y materias

2. ServicioDisponibilidad — CRUD con Validación

- Implementar agregar, listar por profesor, actualizar, eliminar en tabla Disponibilidad_Profesor
- Validación de solapamiento: mismo profesor + mismo día + rangos de tiempo que se intersectan
- Método `obtenerParaSolver()` que retorne QVector
- Conversión de strings de día ("lunes") a int (1)

3. Wiring del Middleware

- Agregar códigos de operación de disponibilidad en messages.h
- Reemplazar stubs de profesor con llamadas reales a ServicioProfesor
- Agregar handlers de disponibilidad (listar, crear, actualizar, eliminar)

Archivos Afectados:

- `src/backend/include/backend/services/ServicioProfesor.hpp` — NUEVO
- `src/backend/src/services/ServicioProfesor.cpp` — NUEVO
- `src/backend/include/backend/services/ServicioDisponibilidad.hpp` — NUEVO
- `src/backend/src/services/ServicioDisponibilidad.cpp` — NUEVO
- `src/middleware/include/middleware/messages.h` — MODIFICADO
- `src/middleware/include/middleware/internalserver.h` — MODIFICADO
- `src/middleware/src/server/internalserver.cpp` — MODIFICADO
- `src/backend/src/services/CMakeLists.txt` — MODIFICADO
- `test/backend/test_servicio_profesor.cpp` — NUEVO
- `test/backend/test_servicio_disponibilidad.cpp` — NUEVO

Criterios de Aceptación:

- ☐ ServicioProfesor CRUD funcionando con validación
- ☐ ServicioDisponibilidad CRUD con detección de solapamiento
- ☐ Middleware: los 5 handlers de profesor usan ServicioProfesor (sin stubs)
- ☐ Middleware: rutas de disponibilidad agregadas
- ☐ Unit tests para ambos servicios
- ☐ Todos los tests existentes pasan sin regresión

Issue #39: Backend — Implementación Restricciones Blandas CP-SAT

Asignado: Luis | **Tipo:** Backend | **Depende de:** Ninguna

Objetivo: Implementar restricciones blandas en el solver CP-SAT. Son restricciones que DEBEN satisfacerse pero PUEDEN violarse a un costo. El solver minimizará la penalización total usando una función objetivo.

Contexto Técnico:

- 8 restricciones duras existentes en 3 archivos
- PenalizacionConfig tiene `capacidad_aula=100` y `emergencia_profesor=50` pero NO es referenciada por el solver
- Solver::resolver() actualmente no tiene función objetivo — encuentra CUALQUIER solución factible
- La tabla Disponibilidad_Profesor EXISTE en la BD
- variables.cpp ya filtra creación de BoolVars por disponibilidad

Requisitos:

1. Restricción Blanda — Preferencia de Disponibilidad del Profesor

- Cuando un profesor ESTÁ disponible para un slot pero PREFIERE no enseñar en ese momento, el solver DEBE asignarlos en otro lugar si es posible
- Penalización cuando el profesor es asignado a un slot no preferido

2. Restricción Blanda — Preferencia de Capacidad del Aula

- Cuando un curso usa un aula QUE ESTÁ CERCA pero NO excediendo capacidad, el solver DEBE preferir un aula más grande si está disponible
- Penalización cuando utilización > 90% de capacidad

3. Restricción Blanda — Balance de Carga del Profesor

- El solver DEBE distribuir horas lo más uniformemente posible en los días disponibles
- Penalización cuando un profesor tiene más de 4 slots consecutivos en un solo día

4. Integración de Función Objetivo

- El solver AGREGA una función Minimize que suma todas las penalizaciones de restricciones blandas
- Cada restricción contribuye: peso × conteo de violaciones

5. Expansión de PenalizacionConfig

- Agregar campos: `penalizacion_disponibilidad_profesor` (default: 50), `penalizacion_capacidad_aula` (default: 100), `penalizacion_turno_excedido` (default: 30)

6. Compatibilidad hacia Atrás

- Todas las restricciones duras existentes permanecen SIN CAMBIOS
- El solver DEBE seguir retornando infeasible si las restricciones duras no pueden satisfacerse

Archivos Afectados:

- `src/backend/include/backend/solver/restricciones/restricciones_blandas.hpp` — NUEVO
- `src/backend/src/solver/restricciones/restricciones_blandas.cpp` — NUEVO

- `src/backend/include/backend/solver/config/solver_config.hpp` — MODIFICADO
- `src/backend/src/solver/config/solver_config.cpp` — MODIFICADO
- `src/backend/src/solver/solver.cpp` — MODIFICADO
- `src/backend/include/backend/solver/solver.hpp` — MODIFICADO
- `test/backend/test_restricciones_blandas.cpp` — NUEVO

Criterios de Aceptación:

- ☐ Nuevo archivo `restricciones_blandas.hpp/cpp` con funciones de restricciones blandas
 - ☐ `PenalizacionConfig` expandido con 3 campos nuevos
 - ☐ Función objetivo agregada al solver cuando existen restricciones blandas
 - ☐ `ResultadoSolver` incluye puntuación de penalización y conteo de violaciones
 - ☐ Las 8 restricciones duras existentes sin cambios y pasando
 - ☐ Unit tests para cada restricción blandas
 - ☐ Sin regresión en tests existentes del solver
-

Issue #37: Backend — Tuning del Modelo Solver CP-SAT

Asignado: Nicole | **Tipo:** Backend | **Depende de:** #39

Objetivo: Ajustar los parámetros del solver CP-SAT y la formulación de restricciones para mejorar la calidad de la solución y la factibilidad para escenarios de horarios escolares reales.

Contexto Técnico:

- Solver ejecuta con parámetros CP-SAT por defecto (sin límites de tiempo explícitos)
- `Solver::resolver()` llama `SolveCpModel` con un `Model` sin parámetros configurados
- Variables: `BoolVar assignment[p][m][c][d][s]`

Requisitos:

1. Límite de Tiempo Configurable

- El solver DEBE aceptar un límite de tiempo configurable (en segundos) via `SolverConfig`
- Default: 30s
- El solver DEBE reportar si se alcanzó el límite de tiempo

2. Configuración de Estrategia de Búsqueda

- El solver DEBE soportar configuración de heurística de búsqueda
- Opciones: `CHOOSE_FIRST`, `CHOOSE_MIN_DOMAIN`, `CHOOSE_RANDOM`
- Default: `CHOOSE_FIRST`

3. Función Objetivo con Penalizaciones

- Cuando existen restricciones blandas (de Issue #39), el solver AGREGA un objetivo que minimiza la suma ponderada de violaciones
- Pesos de `PenalizacionConfig`

4. Métricas de Calidad de Solución

- El solver DEBE reportar: puntuación total de penalización, número de violaciones, tiempo hasta primera solución

Archivos Afectados:

- `src/backend/include/backend/solver/config/solver_config.hpp` — MODIFICADO
- `src/backend/src/solver/config/solver_config.cpp` — MODIFICADO
- `src/backend/include/backend/solver/solver.hpp` — MODIFICADO
- `src/backend/src/solver/solver.cpp` — MODIFICADO
- `test/backend/test_solver_config.cpp` — NUEVO

Criterios de Aceptación:

- ☐ SolverConfig soporta campo `time_limit` (segundos, default 30)
- ☐ SolverConfig soporta campo `search_strategy` (default FIRST)
- ☐ Pesos de PenalizacionConfig usados en función objetivo
- ☐ ResultadoSolver incluye métricas de calidad
- ☐ Todos los tests existentes pasan
- ☐ Nuevos unit tests para parsing de parámetros y métricas

Issue #41: Backend — Optimización y Performance del Solver CP-SAT

Asignado: Nicole | **Tipo:** Backend | **Depende de:** #39

Objetivo: Optimizar el rendimiento del solver CP-SAT para problemas de horarios escolares reales. Reducir tiempo de resolución, manejar instancias más grandes y agregar benchmarks de performance.

Contexto Técnico:

- Variables: BoolVar count crece como $O(\text{profesores} \times \text{materias} \times \text{cursos} \times \text{días} \times \text{slots})$
- Para 50 profesores, 30 materias, 20 cursos, 5 días, 12 slots = 180,000 BoolVars potenciales
- Cada restricción itera sobre todos los BoolVars $\rightarrow O(n^2)$ potencial
- QMap iteración para construcción de restricciones es lento para modelos grandes

Requisitos:

1. Optimización de Creación de Variables

- Pre-filtrar combinaciones inválidas antes de crear BoolVars
- Construir índice de combinaciones válidas primero

2. Optimización de Construcción de Restricciones

- Usar índices pre-construidos en lugar de iterar todos los BoolVars
- QMap key por (profesor, día, slot) para $O(1)$ lookup

3. Parámetros CP-SAT

- `max_time_in_seconds`: desde SolverConfig (default 30)
- `num_workers`: auto (usar todos los cores)
- `log_search_progress`: false (excepto modo debug)

4. Soporte de Resolución Incremental

- Soportar solución previa como hint (warm start) a CP-SAT
- Reduce tiempo de re-resolución después de cambios pequeños

5. Suite de Benchmarks

- Probar en múltiples tamaños: pequeño (10 cursos), mediano (30), grande (50)
- Resultados en archivo JSON para comparación

6. Guard de Regresión de Performance

- Comparar resultados contra baseline
- Si tiempo de resolución regresa >20%, el test FALLA

Archivos Afectados:

- `src/backend/src/solver/variables.cpp` — MODIFICADO
- `src/backend/src/solver/restricciones/restricciones_profesores.cpp` — MODIFICADO
- `src/backend/src/solver/restricciones/restricciones_turnos.cpp` — MODIFICADO
- `src/backend/src/solver/restricciones/restricciones_aulas.cpp` — MODIFICADO
- `src/backend/src/solver/solver.cpp` — MODIFICADO
- `src/backend/include/backend/solver/solver.hpp` — MODIFICADO
- `test/backend/test_solver_benchmark.cpp` — MODIFICADO
- `test/backend/benchmark_baseline.json` — NUEVO

Criterios de Aceptación:

- ☐ Creación de variables usa índices pre-filtrados
- ☐ Construcción de restricciones usa lookups con keys
- ☐ Parámetros CP-SAT configurados (límite de tiempo, workers, logging)
- ☐ Soporte de warm start via hint de solución previa
- ☐ Suite de benchmarks con 3 tamaños de problema
- ☐ Guard de regresión compara contra baseline
- ☐ Todos los tests existentes del solver pasan
- ☐ Nuevos tests de performance en `test_solver_benchmark.cpp`

Issue #36: Frontend — Desarrollo de Módulos de Administración (Gestión de Usuarios)

Asignado: Dani | **Tipo:** Frontend | **Depende de:** ServicioProfesor*

Objetivo: Reemplazar el CRUD de profesores basado en stubs en el frontend con persistencia real vía middleware → backend. Conectar todas las operaciones de profesores (crear, listar, editar, eliminar) y hacer funcional el grid de disponibilidad (no hardcodeado).

Contexto Técnico:

- TeacherListWidget envía mensajes IPC vía InternalClient pero los handlers del middleware son STUBS
- El grid de disponibilidad del panel derecho está hardcodeado con celdas de colores estáticos

- El botón "Gestionar Disponibilidad" no tiene slot conectado
- Después de la tarea de ServicioProfesor: existirá el servicio en backend y los stubs serán reemplazados

Requisitos:

1. Lista de Profesores se Carga desde BD

- TeacherListWidget SOLICITA la lista de profesores vía `OP_LISTA_PROFESORES` cuando se muestra
- La tabla SOLO muestra profesores que existen en la base de datos

2. Crear Profesor Persiste en Full Stack

- El botón "+" abre TeacherFormDialog
- Al guardar, envía `OP_CREAR_PROFESOR` con el payload completo
- En éxito, el nuevo profesor aparece en la tabla
- Validación: campos requeridos no vacíos

3. Editar Profesor Persiste en Full Stack

- Click en ícono de editar abre TeacherFormDialog pre-poblado
- Al guardar, envía `OP_ACTUALIZAR_PROFESOR`
- En éxito, la fila se actualiza

4. Eliminar Profesor con Confirmación

- Click en ícono de eliminar muestra diálogo de confirmación
- Al confirmar, envía `OP_ELIMINAR_PROFESOR`
- En éxito, la fila se elimina

5. Grid de Disponibilidad Muestra Datos Reales

- El grid del panel derecho muestra datos reales de disponibilidad del profesor seleccionado
- Celdas verdes = disponible, 其他 = no disponible

6. Botón "Gestionar Disponibilidad" Abre Editor

- Abre diálogo donde el usuario puede activar/desactivar slots de tiempo
- Los cambios persisten vía IPC

Archivos Afectados:

- `src/frontend/src/views/teacher_list_widget.hpp` — MODIFICADO
- `src/frontend/src/views/teacher_list_widget.cpp` — MODIFICADO
- `src/frontend/src/forms/teacher_form_dialog.hpp` — POSIBLEMENTE SIN CAMBIOS
- `src/frontend/src/forms/availability_dialog.hpp` — NUEVO
- `src/frontend/src/forms/availability_dialog.cpp` — NUEVO

Criterios de Aceptación:

- ☐ Las 4 operaciones CRUD (listar, crear, editar, eliminar) funcionan end-to-end con persistencia
- ☐ La tabla muestra datos de la BD, no de stubs

- ☐ Grid de disponibilidad refleja datos de BD
 - ☐ Botón "Gestionar Disponibilidad" abre diálogo funcional
 - ☐ Validación de formulario previene campos requeridos vacíos
 - ☐ Respuestas de error del backend muestran mensajes amigables
 - ☐ Sin regresión en vistas existentes
-

Issue #38: Frontend — Pantalla de Configuración del Sistema (Shell Estético)

Asignado: Paola | **Tipo:** Frontend | **Depende de:** Ninguna

Objetivo: Crear una ventana de configuración del sistema **puramente estética/general** en el frontend. Es una tarea de scaffolding: la ventana debe ser funcional y navegable con paneles placeholder para configuraciones generales (turnos, recesos, duración de slots, temas visuales y futuros paneles de configuración). **SIN configuración del solver.**

Contexto Técnico:

- MainWindow tiene 6 vistas. NO existe vista de configuración
- Esta sprint solo crea el shell. La implementación de paneles individuales es para sprints futuros
- La configuración del solver se maneja desde archivos de configuración backend, NO desde esta UI

Requisitos:

1. Vista de Configuración Accesible desde Sidebar

- Nuevo ítem "Configuración" en el sidebar
- Al hacer click, muestra el shell de configuración en QStackedWidget

2. Shell con Sidebar/Tabs Secundario

- El shell tiene navegación secundaria (sidebar o tabs) con secciones placeholder:
 - General (turnos, recesos, duración de slots)
 - Visual (temas, colores, fuentes)
 - Futuras configuraciones
- Cada sección muestra un widget placeholder con título y descripción

3. Arquitectura Extensible

- Agregar un nuevo panel = crear widget + registrarlo en el shell
- No se necesitan modificar otros archivos

4. Paneles Placeholder con Etiquetas Claras

- Cada panel muestra: título de sección, descripción breve, texto "Próximamente"

5. Sin Configuración del Solver

- La ventana NO contiene ningún parámetro del solver
- La configuración del solver permanece en archivos de configuración backend

6. Punto de Entrada Limpio

- Shell profesional y expandible sin refactorización mayor

Archivos Afectados:

- `src/frontend/src/views/configuracion_widget.hpp` — NUEVO
- `src/frontend/src/views/configuracion_widget.cpp` — NUEVO
- `src/frontend/src/views/config_placeholder_general.hpp` — NUEVO
- `src/frontend/src/views/config_placeholder_general.cpp` — NUEVO
- `src/frontend/src/views/config_placeholder_visual.hpp` — NUEVO
- `src/frontend/src/views/config_placeholder_visual.cpp` — NUEVO
- `src/frontend/src/views/main_window.hpp` — MODIFICADO
- `src/frontend/src/views/main_window.cpp` — MODIFICADO

Criterios de Aceptación:

- ☐ Nueva vista "Configuración" en sidebar y QStackedWidget
- ☐ ConfigShellWidget con navegación secundaria
- ☐ Paneles placeholder para: General, Visual, al menos uno "Futuro"
- ☐ Cada placeholder tiene título + descripción + texto placeholder
- ☐ SIN configuración del solver en la ventana de configuración
- ☐ Arquitectura extensible: agregar panel = crear widget + registrar
- ☐ UI limpia y profesional

Issue #40: Middleware/QA — Pruebas de Integración Front → Middle → Back + Reporte

Asignado: Manuel | **Tipo:** QA | **Depende de:** ServicioProfesor* + #36

Objetivo: Crear pruebas de integración end-to-end que verifiquen el flujo completo de datos: Frontend IPC → Middleware → Backend Service → SQLite DB → respuesta. También crear un generador de reportes de resultados.

Contexto Técnico:

- Existe `test/test_crud_routes.cpp` con pruebas básicas de rutas
- No existen pruebas de integración que cubran la cadena completa
- Middleware: `InternalServer/QLocalServer` con `QHash route dispatch`
- Backend services: `ServicioAula`, `ServicioMaterias` (implementaciones reales)

Requisitos:

1. Pruebas Middleware-to-Backend

- Verificar que cada handler CRUD delega correctamente al servicio backend y retorna datos reales de la BD
- Spin up `InternalServer`, enviar mensajes IPC, validar respuestas

2. Pruebas CRUD de Aulas

- Full-stack: `create` → `get` → `update` → `delete` vía middleware → `ServicioAula` → `SQLite`

3. Pruebas CRUD de Materias

- Full-stack: create → read → update → delete vía middleware → ServicioMaterias → SQLite

4. Pruebas CRUD de Disponibilidad

- Full-stack: create, read, update, delete vía middleware → ServicioDisponibilidad → SQLite

5. Pruebas de Manejo de Errores

- Payloads inválidos, campos faltantes, errores de BD retornan códigos de error apropiados
- Timeout de middleware (5 segundos)

6. Generador de Reportes de Pruebas

- Script o herramienta que genere reporte resumen: total de pruebas, pasaron, fallaron, cobertura de operaciones CRUD, y tiempo

7. Manejo de Timeout

- Verificar que el timeout de 5 segundos del middleware funciona correctamente

Archivos Afectados:

- `test/integration/test_teacher_crud_integration.cpp` — NUEVO
- `test/integration/test_classroom_crud_integration.cpp` — NUEVO
- `test/integration/test_subject_crud_integration.cpp` — NUEVO
- `test/integration/test_availability_integration.cpp` — NUEVO
- `test/integration/test_error_handling_integration.cpp` — NUEVO
- `test/integration/test_integration_helper.hpp` — NUEVO
- `test/integration/CMakeLists.txt` — NUEVO
- `scripts/generate_test_report.py` — NUEVO

Criterios de Aceptación:

- ☐ Archivo de pruebas de integración cubre los 5 dominios CRUD
- ☐ Cada dominio tiene al menos: crear, listar, obtener, actualizar, eliminar, escenarios de error
- ☐ Pruebas usan BD real (en memoria o archivo temporal), no servicios mockeados
- ☐ Reporte generado en formato markdown
- ☐ Todas las pruebas de integración pasan
- ☐ Prueba de timeout valida el timeout de 5 segundos del middleware
- ☐ Pruebas documentadas con comentarios given/when/then

Desglose de Tareas

Fase 1: Foundation — ServicioProfesor + ServicioDisponibilidad (Luis)

Tarea	Descripción	Archivos	Esfuerzo
T-1.1	Crear ServicioProfesor.hpp — CRUD + validación + Profesor_Materia + obtenerTodosParaSolver	ServicioProfesor.hpp	M

Tarea	Descripción	Archivos	Esfuerzo
T-1.2	Crear ServicioProfesor.cpp — implementar CRUD contra tabla Profesores (TEXT PK), cascade delete	ServicioProfesor.cpp	L
T-1.3	Crear ServicioDisponibilidad.hpp — CRUD + validación solapamiento + obtenerParaSolver	ServicioDisponibilidad.hpp	M
T-1.4	Crear ServicioDisponibilidad.cpp — implementar contra tabla Disponibilidad_Profesor	ServicioDisponibilidad.cpp	L
T-1.5	Agregar constantes OP de disponibilidad a messages.h	messages.h	S
T-1.6	Agregar declaraciones de handlers de disponibilidad a internalserver.h	internalserver.h	S
T-1.7	Reemplazar stubs de profesor en internalserver.cpp con ServicioProfesor real	internalserver.cpp	M
T-1.8	Implementar handlers de disponibilidad en internalserver.cpp	internalserver.cpp	M
T-1.9	Actualizar src/backend/CMakeLists.txt — agregar nuevos archivos fuente	CMakeLists.txt	S
T-1.10	Escribir test_servicio_profesor.cpp — unit tests	test_servicio_profesor.cpp	M
T-1.11	Escribir test_servicio_disponibilidad.cpp — unit tests	test_servicio_disponibilidad.cpp	M
T-1.12	Actualizar test/CMakeLists.txt — agregar nuevos targets de test	CMakeLists.txt	S

Fase 2: Restricciones Blandas — Expansión PenalizacionConfig (Luis)

Tarea	Descripción	Archivos	Esfuerzo
T-2.1	Expandir PenalizacionConfig en solver_config.hpp — 3 campos nuevos	solver_config.hpp	S
T-2.2	Actualizar solver_config.cpp — parsear 3 campos nuevos con defaults	solver_config.cpp	S
T-2.3	Crear restricciones_blandas.hpp — declarar agregarRestriccionesBlandas	restricciones_blandas.hpp	S
T-2.4	Crear restricciones_blandas.cpp — implementar 3 restricciones de penalización	restricciones_blandas.cpp	L
T-2.5	Conectar en solver.cpp — agregar llamada a agregarRestriccionesBlandas + Maximize	solver.cpp	M

Tarea	Descripción	Archivos	Esfuerzo
T-2.6	Actualizar src/backend/CMakeLists.txt — agregar restricciones_blandas.cpp	CMakeLists.txt	S
T-2.7	Escribir test_restricciones_blandas.cpp — unit tests	test_restricciones_blandas.cpp	M

Fase 3: Tuning + Performance del Solver (Nicole)

Tarea	Descripción	Archivos	Esfuerzo
T-3.1	Agregar CpSolverParameters a Solver::resolver() — max_time_in_seconds, num_workers	solver.cpp	M
T-3.2	Agregar tiempo_limite_s a GeneracionConfig — parsear de JSON	solver_config.hpp/cpp	S
T-3.3	Agregar lookup maps a VariablesSolver — QMap por (profesor, día)	variables.hpp/cpp	M
T-3.4	Refactorizar restricciones_profesores.cpp — usar lookup maps	restricciones_profesores.cpp	M
T-3.5	Escribir test_solver_benchmark_extended.cpp — benchmarks	test_solver_benchmark_extended.cpp	M
T-3.6	Ajustar CpSolverParameters iterativamente — documentar valores óptimos	solver.cpp	S

Fase 4: Frontend Admin — Gestión de Profesores (Dani)

Tarea	Descripción	Archivos	Esfuerzo
T-4.1	Actualizar teacher_list_widget — conectar a rutas reales del middleware	teacher_list_widget.cpp	M
T-4.2	Actualizar teacher_form_dialog — conectar a OP_CREAR/ACTUALIZAR_PROFESOR	teacher_form_dialog.cpp	M
T-4.3	Agregar diálogo de confirmación de eliminación	teacher_list_widget.cpp	S
T-4.4	Agregar sub-vista de disponibilidad — tabla con add/edit/delete	availability_dialog.cpp	M
T-4.5	Agregar sub-vista de asignación de materias	teacher_list_widget.cpp	M
T-4.6	Verificar todas las operaciones CRUD del frontend funcionan end-to-end	—	S

Fase 5: Config Shell — Placeholder Estético (Paola)

Tarea	Descripción	Archivos	Esfuerzo
T-5.1	Crear configuracion_widget.hpp/cpp — QStackedWidget con sidebar	configuracion_widget.hpp/cpp	M
T-5.2	Crear paneles placeholder — GeneralPanel, VisualPanel, FuturePanel	config_placeholder_*.hpp/cpp	M
T-5.3	Registrar botón de configuración en main_window	main_window.hpp/cpp	S
T-5.4	Estilizar el shell — coincidir con el estilo existente de la app	configuracion_widget.cpp	S
T-5.5	Verificar que el shell abre, navega y retorna a vistas principales	—	S

Fase 6: Pruebas de Integración (Manuel)

Tarea	Descripción	Archivos	Esfuerzo
T-6.1	Escribir test_teacher_crud_integration.cpp — end-to-end teacher CRUD	test_teacher_crud_integration.cpp	L
T-6.2	Escribir test_availability_integration.cpp — end-to-end availability	test_availability_integration.cpp	M
T-6.3	Escribir test_classroom_crud_integration.cpp — end-to-end classroom	test_classroom_crud_integration.cpp	M
T-6.4	Escribir test_subject_crud_integration.cpp — end-to-end subjects	test_subject_crud_integration.cpp	M
T-6.5	Generar reporte JUnit XML — configurar test runner	generate_test_report.py	S
T-6.6	Verificar que todos los tests existentes siguen pasando	—	S

Fase 7: Build + Polish

Tarea	Descripción	Archivos	Esfuerzo
T-7.1	Actualizar src/backend/CMakeLists.txt — asegurar todas las fuentes nuevas	CMakeLists.txt	S
T-7.2	Actualizar src/frontend/CMakeLists.txt — agregar configuracion_widget	CMakeLists.txt	S
T-7.3	Actualizar test/CMakeLists.txt — agregar todos los targets nuevos	CMakeLists.txt	S

Tarea	Descripción	Archivos	Esfuerzo
T-7.4	Verificación de build completo — cmake --build sin warnings ni errores	—	M
T-7.5	Ejecutar suite completa de tests — todos pasan, sin regresiones	—	M

Resumen de Tareas

Fase	Tareas	Asignado	Depende de
1. Foundation	12	Luis	—
2. Restricciones Blandas	7	Luis	—
3. Tuning + Performance	6	Nicole	Fase 2
4. Frontend Admin	6	Dani	Fase 1
5. Config Shell	5	Paola	—
6. Pruebas Integración	6	Manuel	Fases 1 + 4
7. Build + Polish	5	Todos	Todas
Total	47		

Camino Crítico

```
Fase 1 (Luis) → Fase 4 (Dani) → Fase 6 (Manuel)
Fase 2 (Luis) → Fase 3 (Nicole)
Fase 5 (Paola) → independiente
```

Oportunidades de Paralelismo

- **Luis** puede trabajar en Fase 1 y Fase 2 en paralelo
- **Paola** (Fase 5) es totalmente independiente — puede empezar en cualquier momento
- **Nicole** (Fase 3) empieza cuando Fase 2 termina
- **Dani** (Fase 4) empieza cuando Fase 1 termina (middleware wiring completado)
- **Manuel** (Fase 6) empieza cuando Fase 1 + Fase 4 terminan

Cambios de Archivos

Capa	Archivos Nuevos	Archivos Modificados
Backend Services	3 (ServicioProfesor, ServicioDisponibilidad, restricciones_blandas)	5 (solver.cpp, solver_config.hpp/cpp, CMakeLists)

Capa	Archivos Nuevos	Archivos Modificados
Solver	1 (restricciones_blandas)	4 (variables.cpp, 3× restricciones_*.cpp)
Middleware	0	3 (messages.h, internalserver.h/cpp)
Frontend	4 (configuracion_widget, 3× config_placeholders)	3 (teacher_list_widget, main_window, CMakeLists)
Tests	8 (unit + integration)	1 (test_solver_benchmark)
Total	16	16

Riesgos

Riesgo	Probabilidad	Mitigación
Tuning del solver requiere datos realistas que no existen aún	Alta	Usar dataset sintético de fixtures existentes; ajustar iterativamente
Refactorización del middleware rompe conexiones existentes del frontend	Media	Mantener op-codes antiguos funcionando junto con nuevos durante transición
Pruebas de integración inestables por IPC asíncrono	Media	Usar cliente síncrono, timeouts generosos, BD aislada por test
Complejidad UI del grid de disponibilidad excede estimado	Baja	Simplificar a vista semanal primero, edición de slots detallada diferida
Scope creep del config shell	Media	Estrictamente limitado a shell/navegación; paneles individuales diferidos

Plan de Rollback

- Todos los servicios nuevos son aditivos (archivos nuevos, sin modificaciones a contratos de servicio existentes)
- Los stubs del middleware pueden restaurarse revirtiendo cambios en internalserver.cpp
- Los cambios del solver están detrás de feature flags (si PenalizacionConfig.soft_enabled == false, comportamiento antiguo preservado)
- Las vistas del frontend son aditivas (nuevo SystemConfigWidget puede eliminarse del sidebar sin romper otras)
- Base de datos: sin cambios de schema necesarios (Disponibilidad_Profesor ya existe)

Criterios de Éxito

- ☐ ServicioProfesor CRUD testeado: crear, leer, actualizar, eliminar profesores vía middleware
- ☐ ServicioDisponibilidad: datos de disponibilidad persisten y cargan correctamente
- ☐ Handlers de middleware para profesor/aula/materia retornan datos reales de BD (sin stubs)
- ☐ Grid de disponibilidad del frontend muestra datos reales y permite edición

- ☐ Restricciones blandas producen puntuaciones de penalización no triviales en datos de prueba
- ☐ Solver completa dentro de 30s en un problema de 20 profesores, 30 cursos
- ☐ Ventana de config shell navegable con paneles placeholder
- ☐ Ventana de config SIN configuración del solver — puramente estética
- ☐ Ventana de config extensible — agregar panel = crear widget + registrar en shell
- ☐ Pruebas de integración cubren al menos: round-trip teacher CRUD, round-trip availability, solver con datos reales